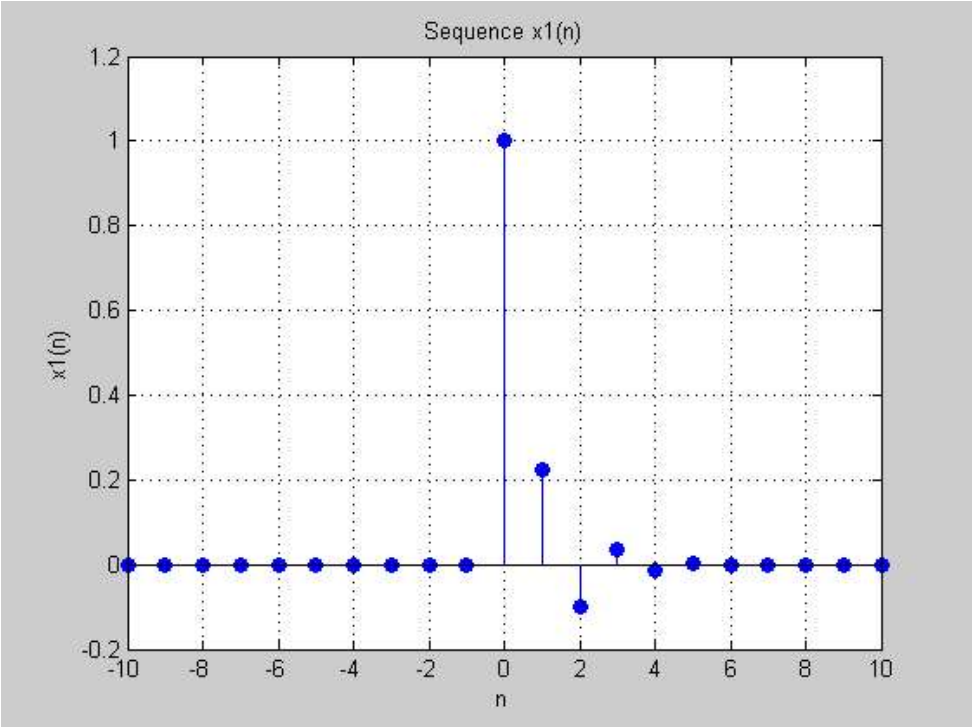
```
clc;clear all;close all;
n = -10:10;

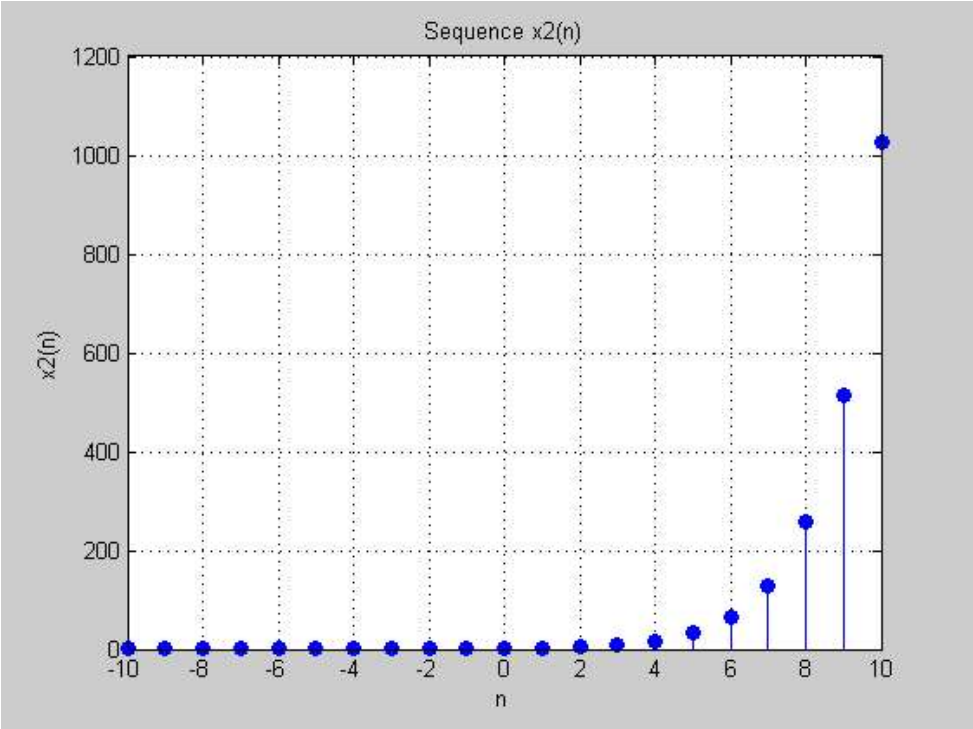
% (a) x1(n)
x1 = (-1/9).^n .* (n >= 0) - (-1/3).^n .* (n >= 1);
% (b) x2(n)
x2 = 1 + 2.^n .* (n >= 1);
% (c) x3(n)
x3 = (0.2).^n .* (n >= 1);

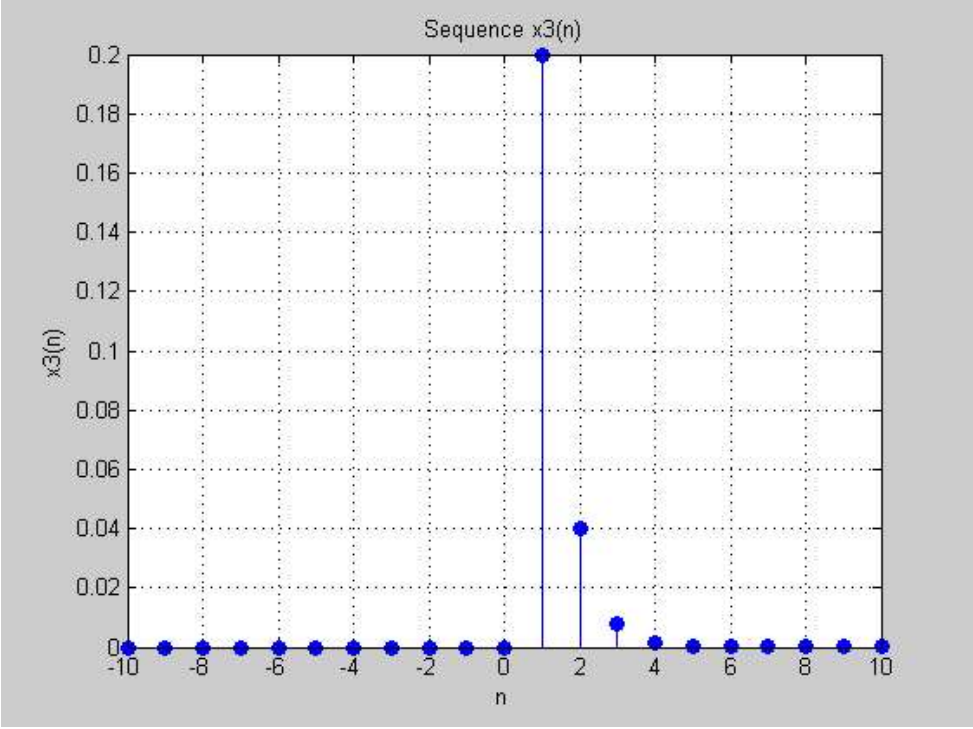
figure;stem(n, x1, 'filled');xlabel('n');ylabel('x1(n)');
title('Sequence x1(n)');grid on;

figure;stem(n, x2, 'filled');xlabel('n');ylabel('x2(n)');
title('Sequence x2(n)');
grid on;

figure;stem(n, x3, 'filled');xlabel('n');ylabel('x3(n)');
title('Sequence x3(n)');grid on;
```



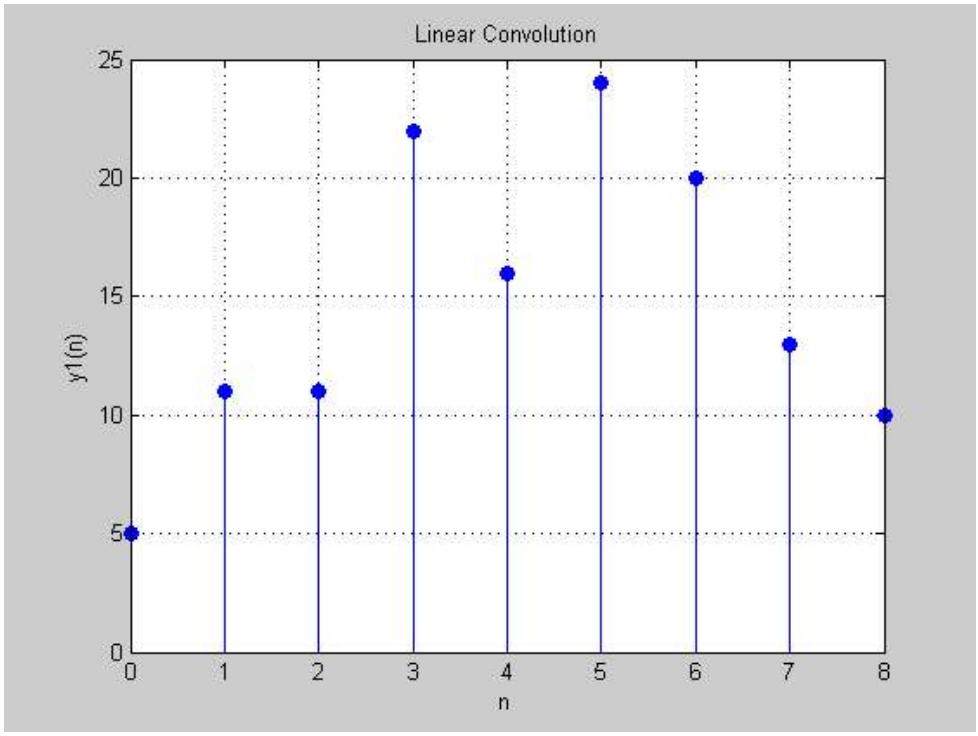


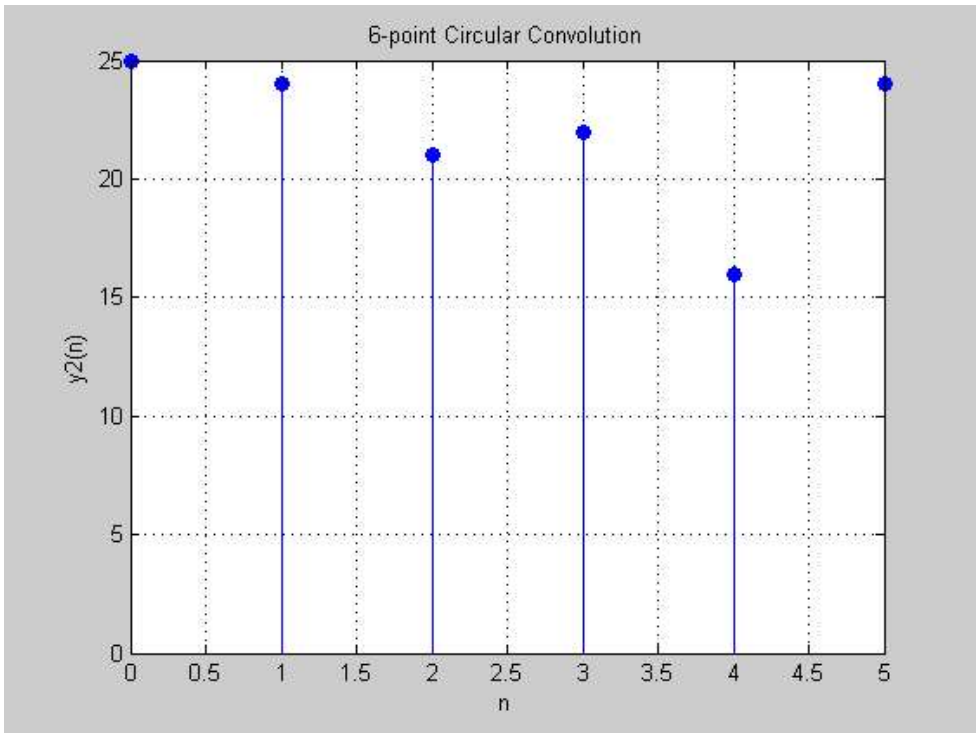


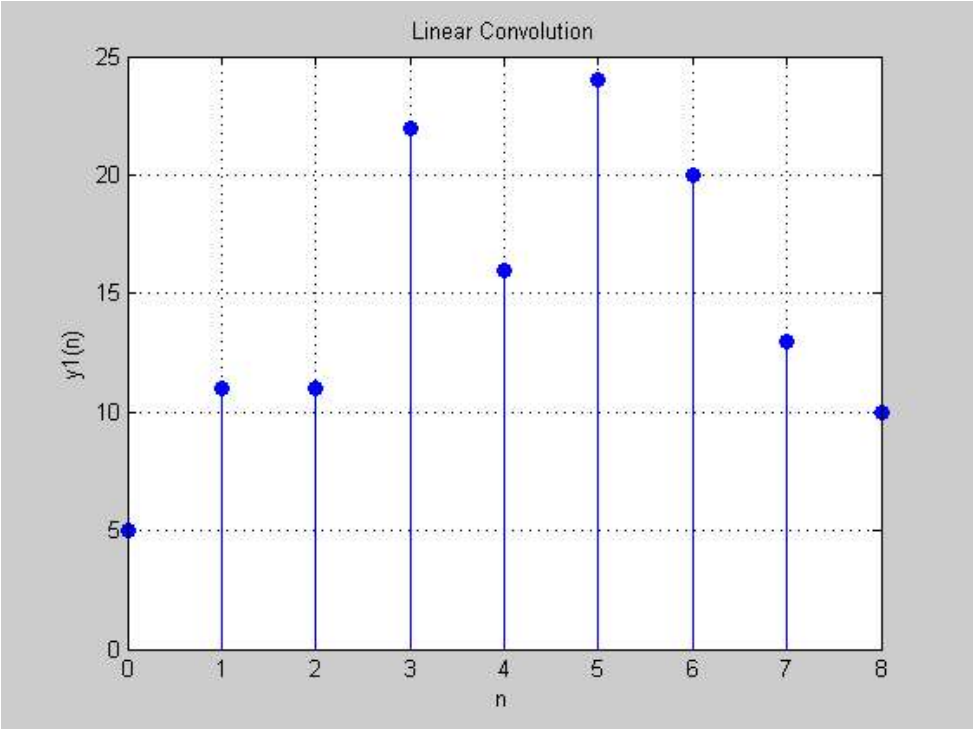

```
%2) Let x1(n)={1?,2,1,2} & x2(n)={5?,4,3,4,5} , X1(k) & X2(k) are the DFTs of x1(n) & x2(n)
%   respectively. Evaluate the linear convolution and 6-point circular convolution using X1(k) & X2(k).
clc;clear;close all;
x1 = [1 2 1 2];
x2 = [5 1 4 3 4 5];
y1 = conv(x1,x2);
N = 6;
y2 = cconv(x1,x2,N);
figure;stem(0:length(y1)-1, y1, 'filled');xlabel('n');ylabel('y1(n)');
title('Linear Convolution');grid on;
figure;stem(0:length(y2)-1, y2, 'filled');xlabel('n');ylabel('y2(n)');
title('6-point Circular Convolution');grid on;
```

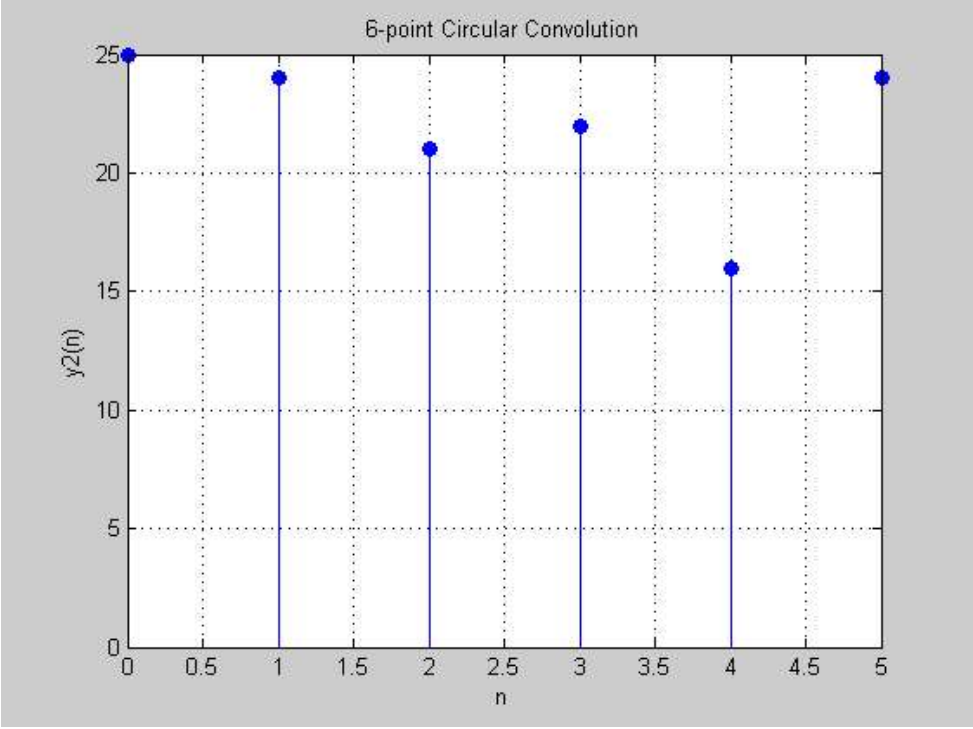
```
x1 = [1 2 1 2];
x2 = [5 1 4 3 4 5];
N = length(x1) + length(x2) - 1;
X1 = fft(x1, N);
X2 = fft(x2, N);
y1 = ifft(X1 .* X2);

N = 6;
X1 = fft(x1, N);
X2 = fft(x2, N);
y2 = ifft(X1 .* X2);
figure;stem(0:length(y1)-1, y1, 'filled');xlabel('n');ylabel('y1(n)');
title('Linear Convolution');grid on;
figure;stem(0:length(y2)-1, y2, 'filled');xlabel('n');ylabel('y2(n)');
title('6-point Circular Convolution');grid on;
```










```
%3). The first six values of the 10-point DFT of a real-valued sequence x(n) are given by
%   X(k) = [10, -2+3j, 3+4j, 2-3j, 4+5j, 12] . Evaluate x(n) and observe magnitude & angle plots of the s

clc;clear;close all;

X = [10, -2+3j, 3+4j, 2-3j, 4+5j, 12];
X = [X, conj(X(5:-1:2))];

x = ifft(X);
disp('Complete 10-point DFT X(k):');
disp(X);
disp('Original sequence x(n):');
disp(real(x));
k = 0:9;
n = 0:9;
```

```
% Magnitude spectrum
figure;stem(k, abs(X), 'filled');xlabel('Frequency index k');ylabel('|X(k)|');
title('Magnitude Spectrum');grid on;

% Phase spectrum
figure;stem(k, angle(X), 'filled');xlabel('Frequency index k');ylabel('Phase (radians)');
title('Phase Spectrum');grid on;

% Original sequence
figure;stem(n, real(x), 'filled');xlabel('n');ylabel('x(n)');
title('Original Sequence');grid on;

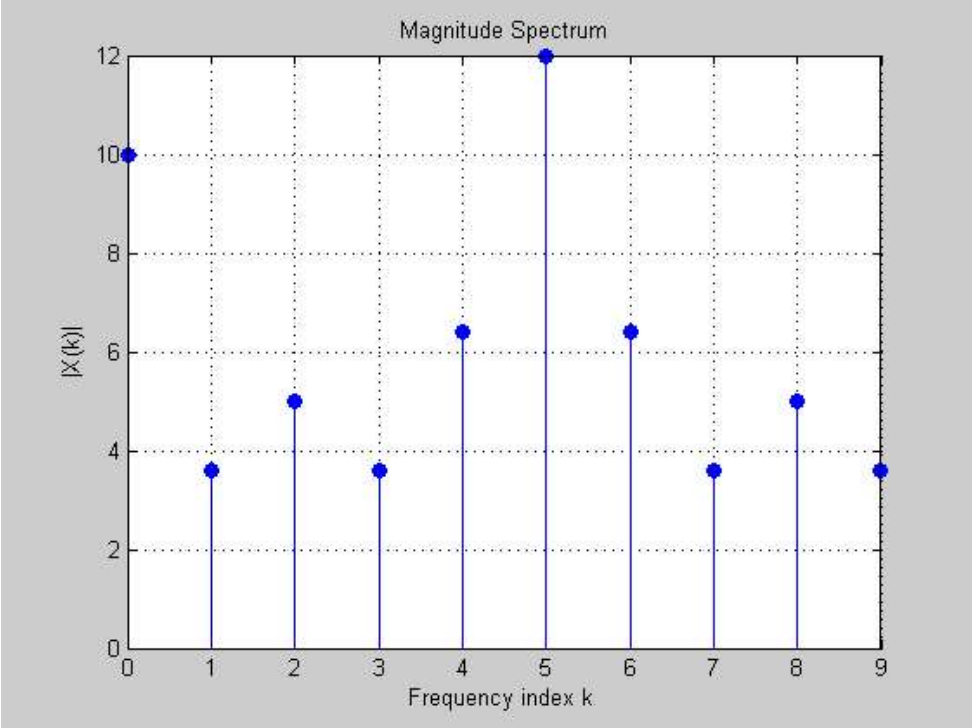

Complete 10-point DFT X(k):
  Columns 1 through 7

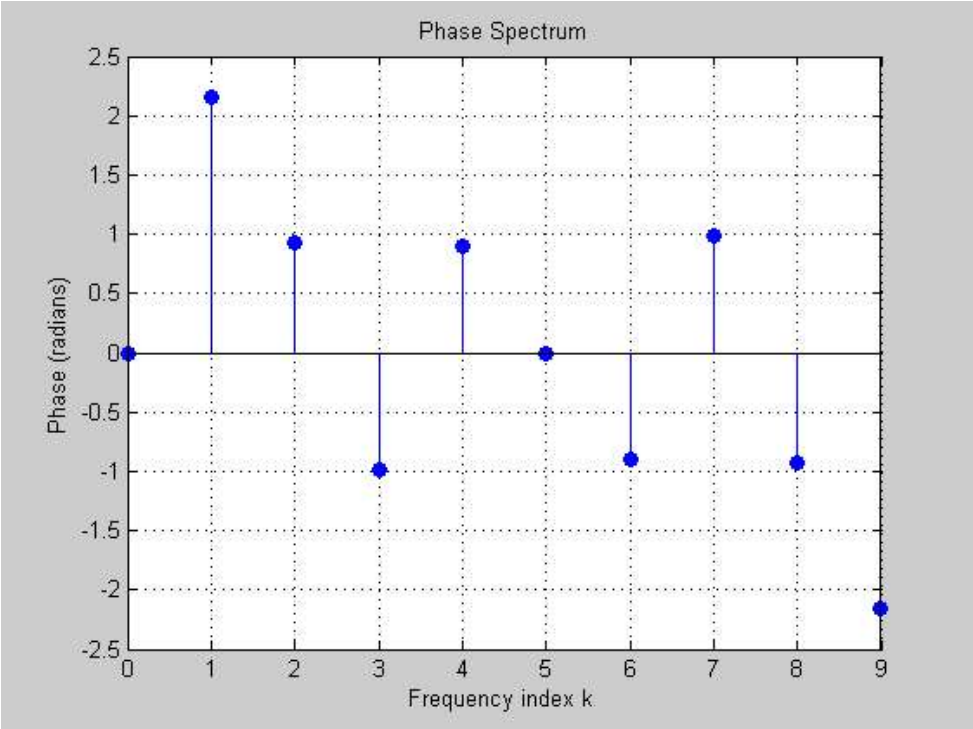
  10.0000            -2.0000 + 3.0000i   3.0000 + 4.0000i   2.0000 - 3.0000i   4.0000 + 5.0000i  12.0000

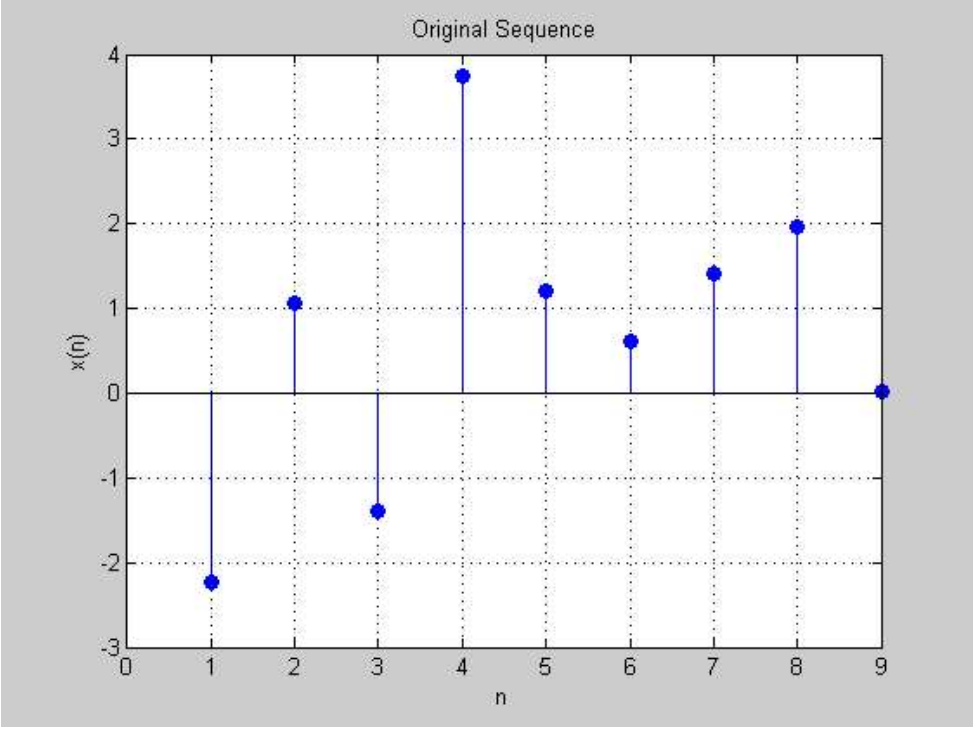
  Columns 8 through 10

   2.0000 + 3.0000i   3.0000 - 4.0000i  -2.0000 - 3.0000i

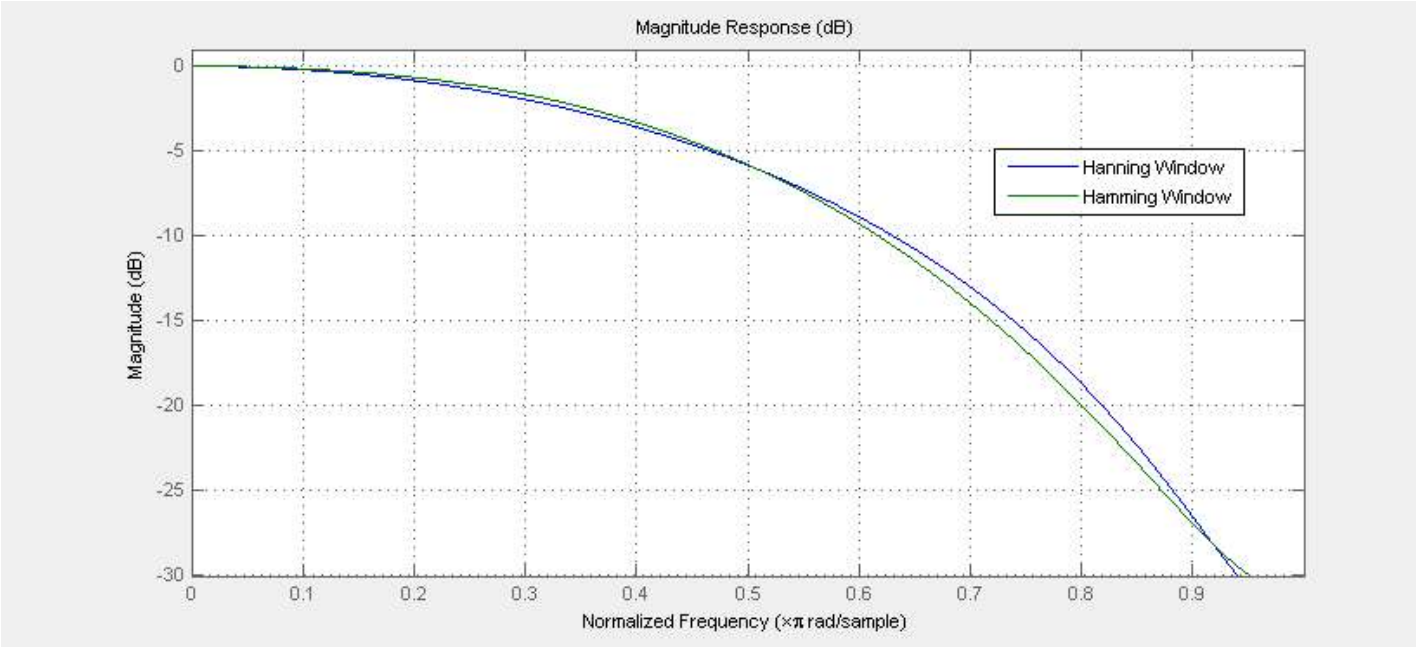
Original sequence x(n):
    3.6000   -2.2397    1.0721   -1.3951    3.7520    1.2000    0.6188    1.4132    1.9571    0.0217
```





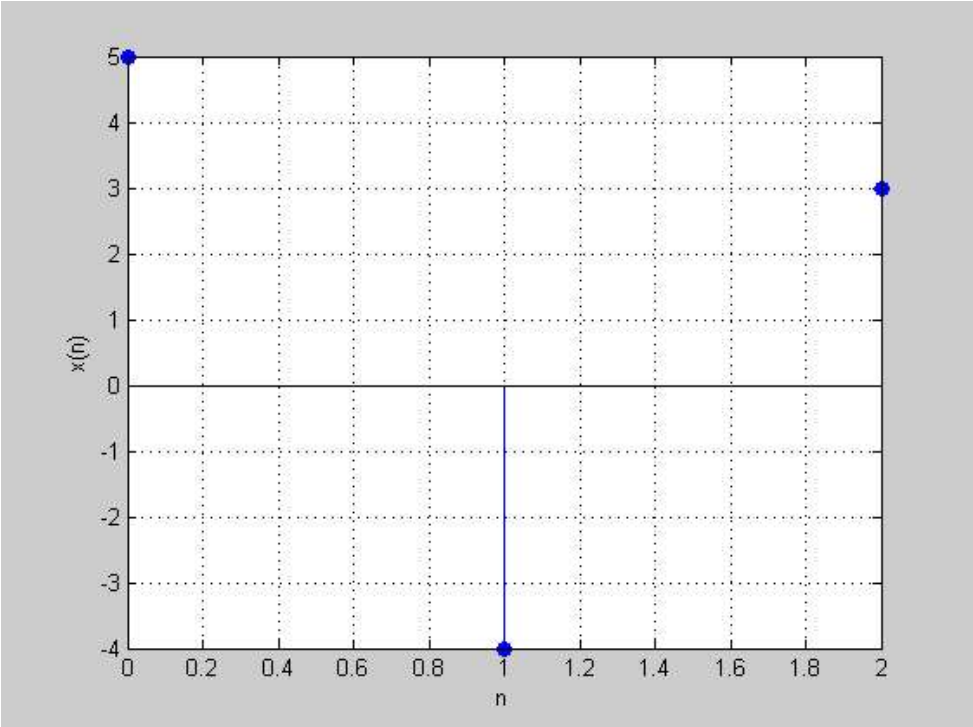


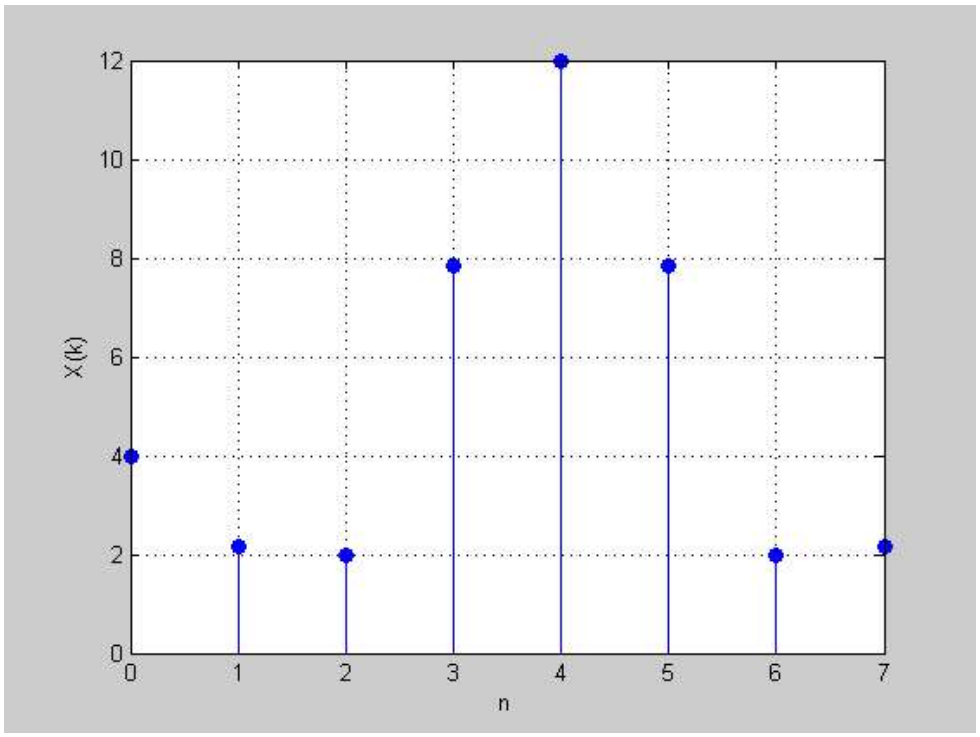

```
%4. Design FIR lowpass filter with desired amplitude response having unity for 0 ? f ? 250Hz and it is
%   zero elsewhere in the range 0 ? f ? f0 ; the sampling frequency 1KHz. Use Hamming and Hanning
%   window function of length 7
clc;clear;close all;
N = 7;
fs = 1000;
fc = 250;
Wc = 2*pi*fc/fs;
h1 = fir1(N-1, Wc/pi, hann(N));
h2 = fir1(N-1, Wc/pi, hamming(N));
H = fvtool(h1, 1, h2, 1);
legend(H, 'Hanning Window', 'Hamming Window');
```

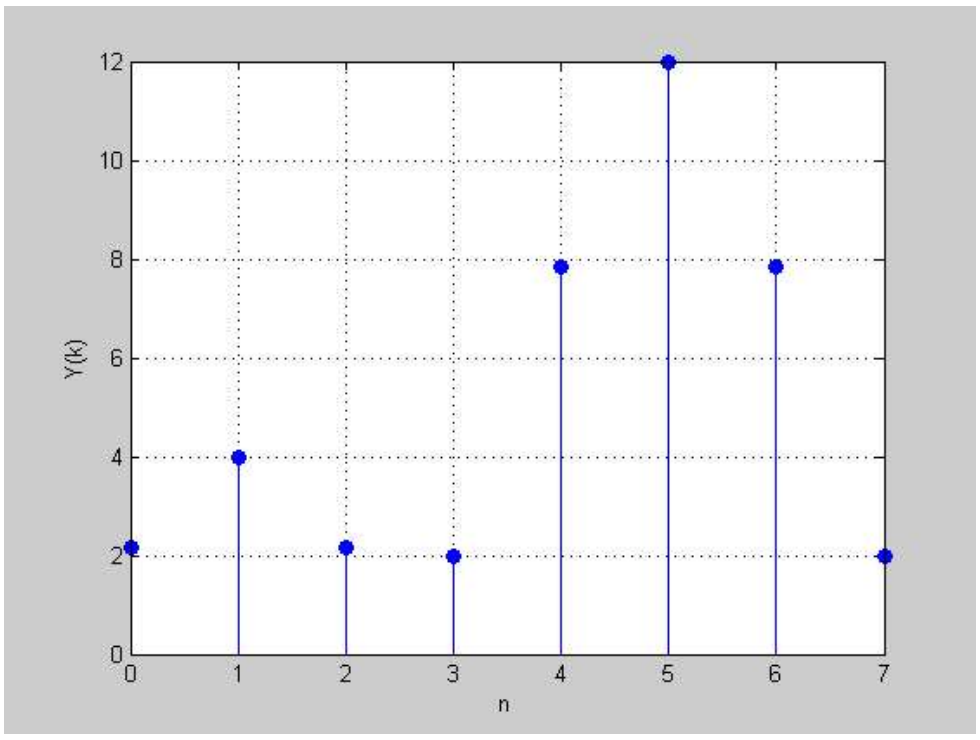


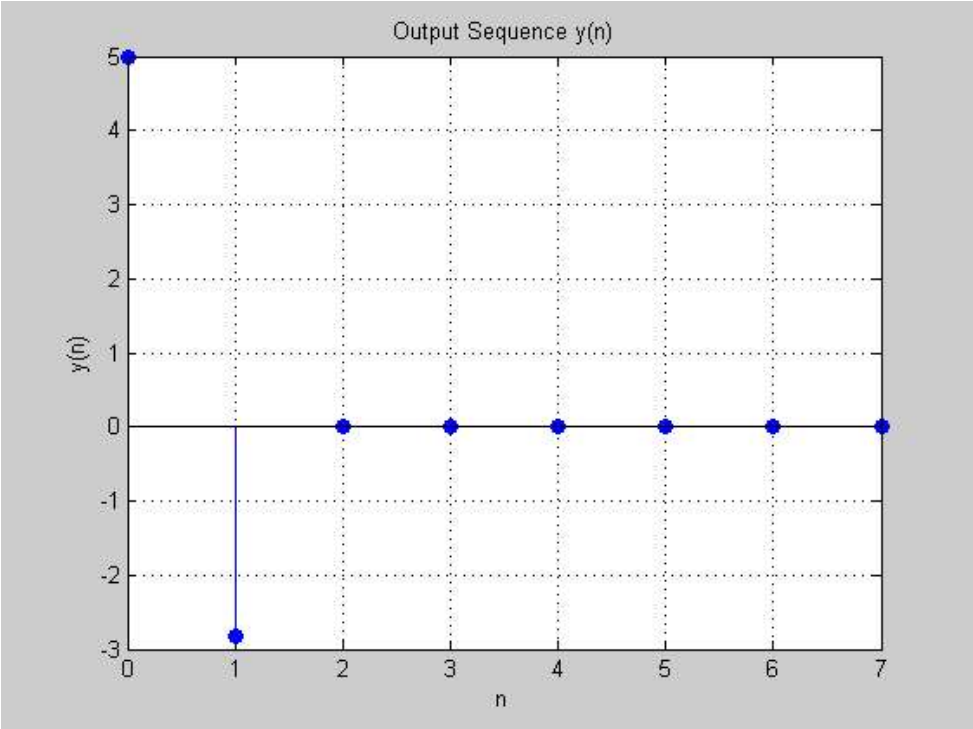

```
%5) Let X(k) be the 8-point DFT of a 3-point sequence x(n) = {5 ? , ?4, 3}. Let Y (k) be the 8-point
%   DFT of a sequence y(n). Determine y(n) when Y(k) = X(k ? 1)8

clc;clear all;close all;
x = [5 -4 3];
X = fft(x,8);
Y = circshift(X,[0 1]);
y = ifft(Y,8);
n = 0:7;
figure;stem(0:length(x)-1, real(x), 'filled');xlabel('n');ylabel('x(n)');grid on;
figure;stem(n, real(X), 'filled');xlabel('n');ylabel('X(k)');grid on;
figure;stem(n, real(Y), 'filled');xlabel('n');ylabel('Y(k)');grid on;
figure;stem(n, real(y), 'filled');xlabel('n');ylabel('y(n)');
title('Output Sequence y(n)');grid on;
```




X(k)
n
0
2
4
6
8
10
12
0
1
2
3
4
5
6
7

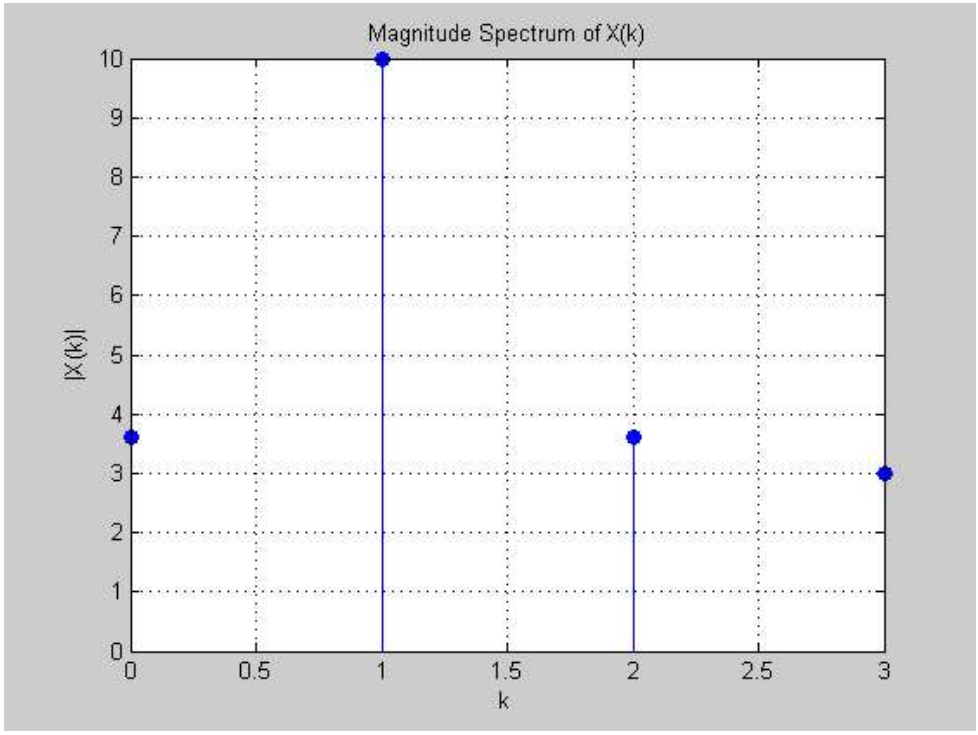

Y(k)
n
0
2
4
6
8
10
12
0
1
2
3
4
5
6
7

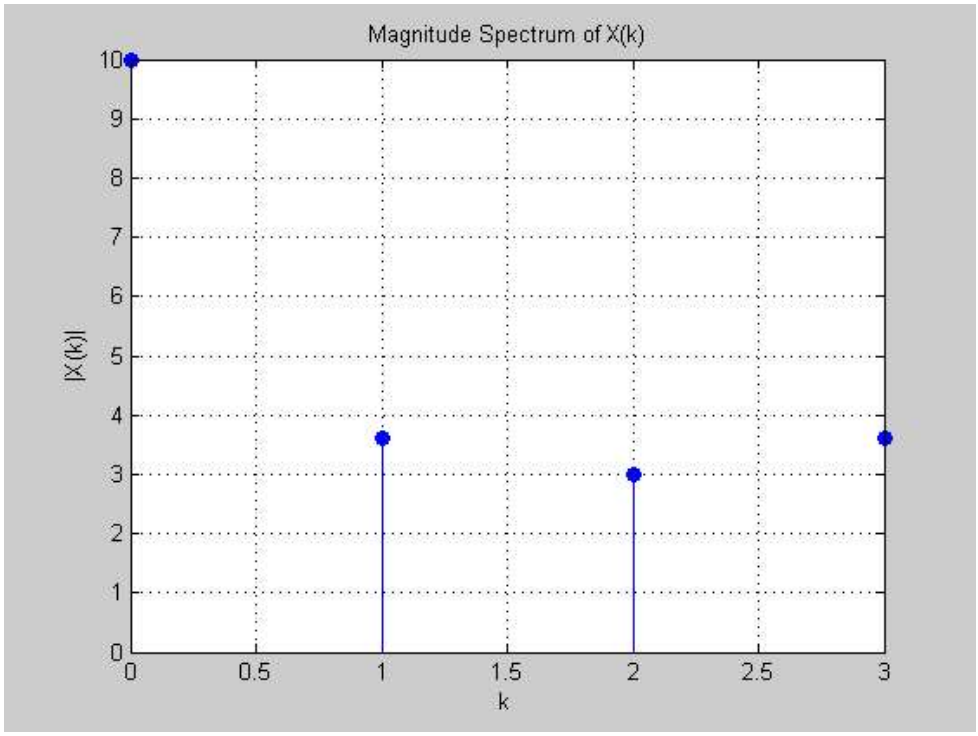


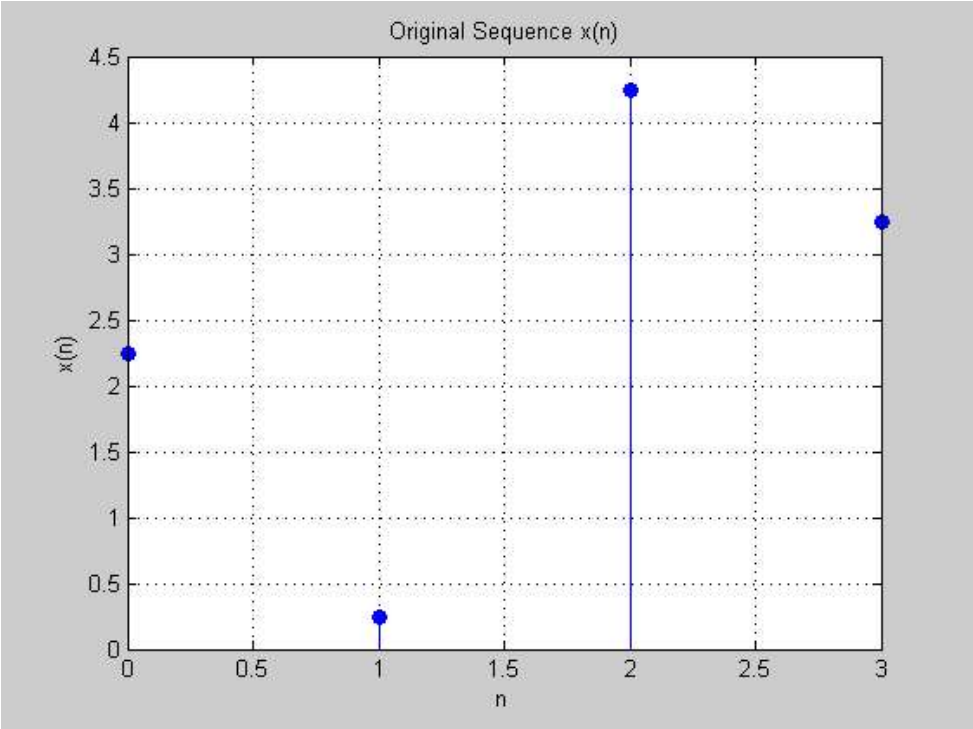

```
%6) Let X(k) be the 4-DFT of a x(n). Given X(k-1)4 = [-2-3j , 10, -2+3j, 3] . Evaluate x(n) and observe
%   magnitude & angle plots of the spectrum X(k)4
clc;clear all;close all;
Y = [-2-3j, 10, -2+3j, 3];
X = circshift(Y, [0 -1]);
x = ifft(X, 4);
k = 0:3;
n = 0:3;

% Magnitude spectrum
figure;
stem(k, abs(Y), 'filled');
xlabel('k');
ylabel('|X(k)|');
title('Magnitude Spectrum of X(k)');
grid on;

figure;
stem(k, abs(X), 'filled');
xlabel('k');
ylabel('|X(k)|');
title('Magnitude Spectrum of X(k)');
grid on;
% Plot x(n)
figure;
stem(n, real(x), 'filled');
xlabel('n');
ylabel('x(n)');
title('Original Sequence x(n)');
grid on;
```


Magnitude Spectrum of X(k)
|X(k)|
k
0
0.5
1
1.5
2
2.5
3
0
1
2
3
4
5
6
7
8
9
10


Magnitude Spectrum of X(k)
|X(k)|
k
0
0.5
1
1.5
2
2.5
3
0
1
2
3
4
5
6
7
8
9
10




```
%7. Design an IIR Butterworth Digital Filter with following specifications using impulse invariance
%   Method with sampling time 1sec. Obtain transfer function, frequency response and pole -zero plot of d
%   a. 3dB attenuation in passband , 0 ? ? ? 0.35?
%   b. 35dB attenuation in stopband 0.45? ? ? ? ?,

clc;
clear;
close all;

T = 1;
fs = 1/T;
wp = 0.35*pi;
ws = 0.45*pi;
Ap = 3;
As = 35;

Wp = wp/T;
Ws = ws/T;

[N,Wn] = buttord(Wp,Ws,Ap,As,'s');

[b,a] = butter(N,Wn,'s');
H1 = tf(b,a);

figure;
freqs(b,a);
title('Analog Butterworth Frequency Response');
grid on;

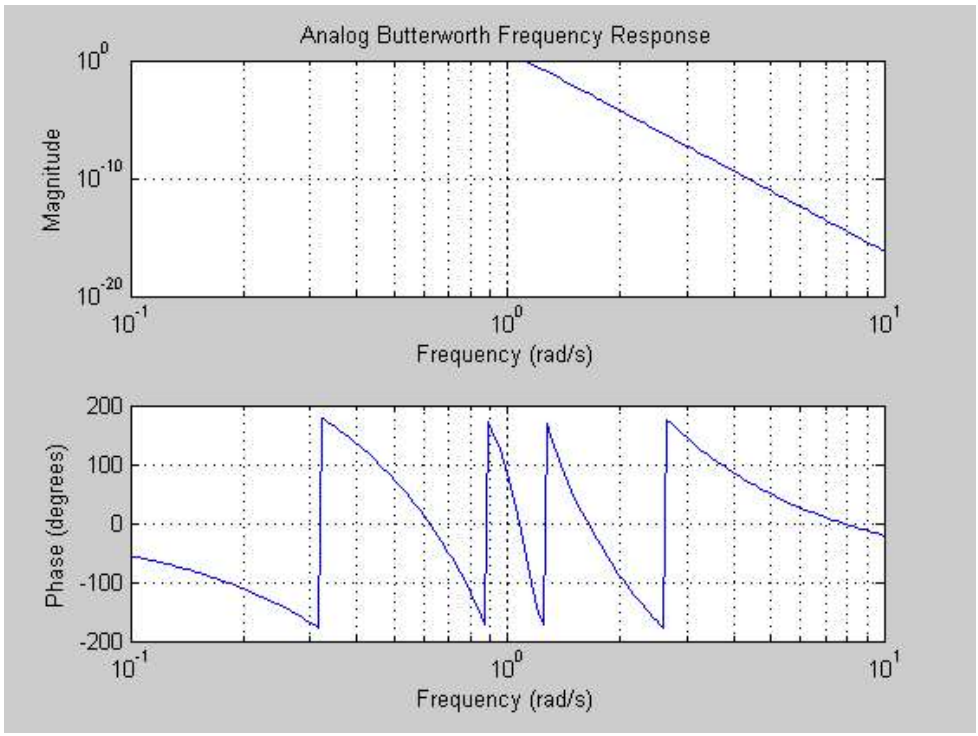
[bz,az] = impinvar(b,a,fs);
H2 = tf(bz,az,T);

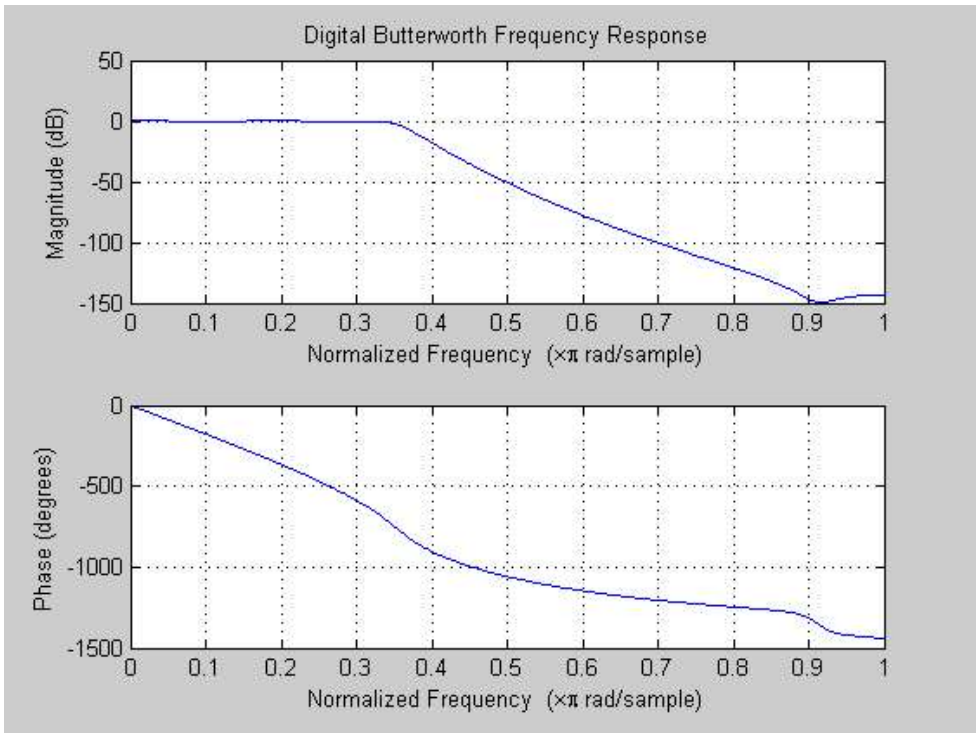
% Digital frequency response
figure;
freqz(bz,az,1024);
title('Digital Butterworth Frequency Response');
grid on;

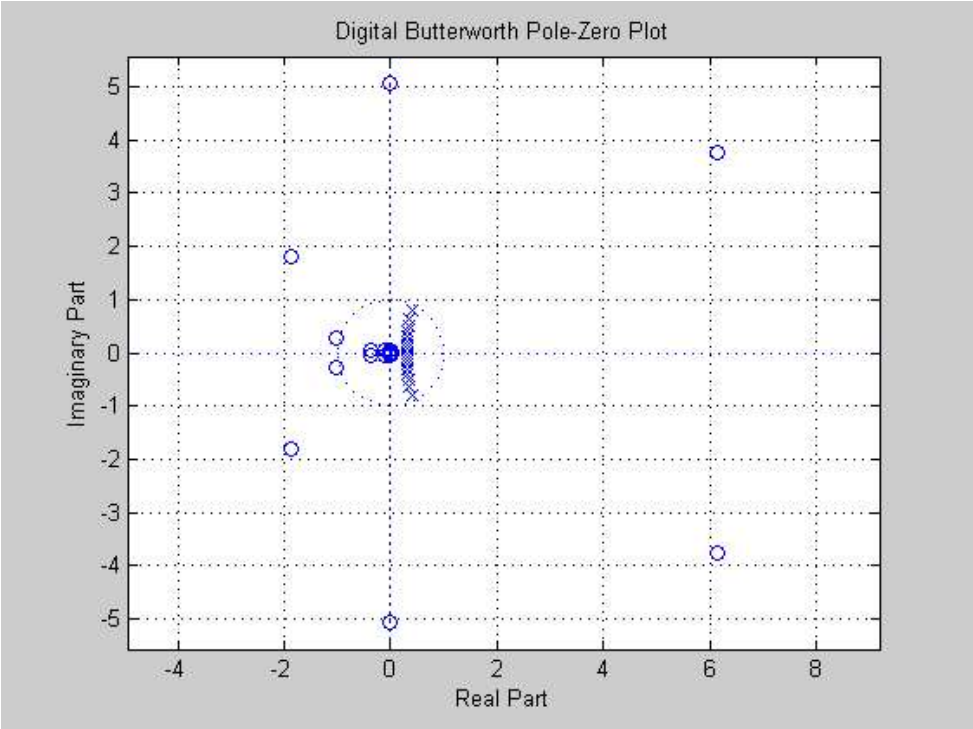
% Digital pole-zero plot
figure;
zplane(bz,az);
title('Digital Butterworth Pole-Zero Plot');
grid on;


Warning:   The output is not correct/robust.
```

```
Coeffs of B(s)/A(s) are real, but B(z)/A(z) has complex coeffs.
Probable cause is rooting of high-order repeated poles in A(s).
```








```
%8. Perform 4-factor interpolation & 3-factor decimation on a sum of sinusoidal sequences having
%   frequencies 0.05Hz & 0.075Hz

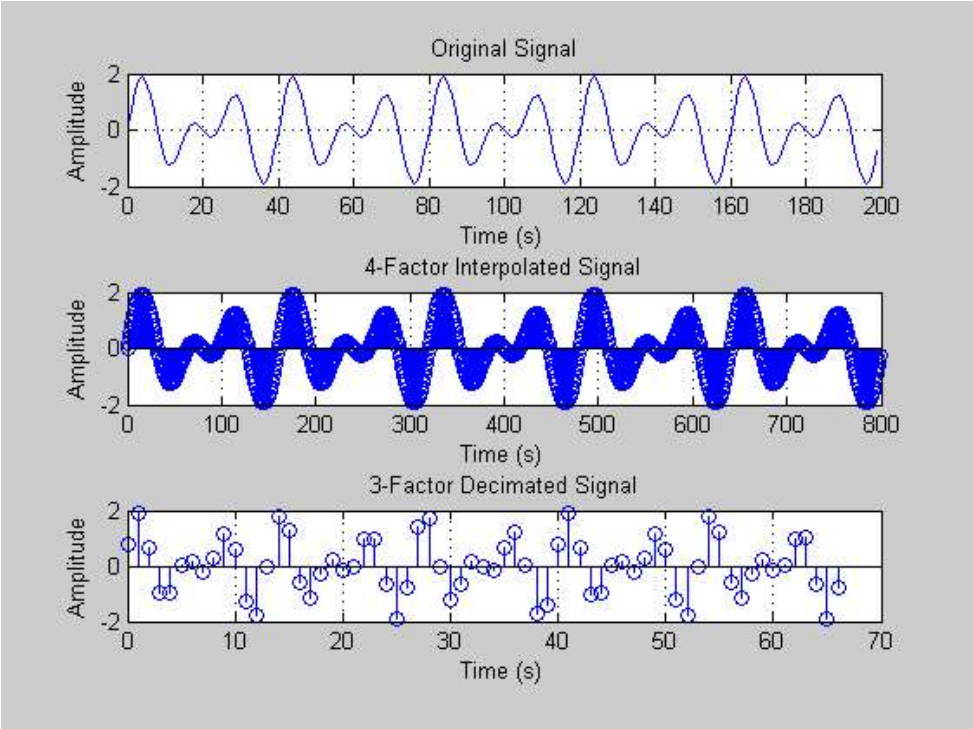
clc;
clear;
close all;

Fs = 1;
t = 0:1/Fs:199;
x = sin(2*pi*0.05*t) + sin(2*pi*0.075*t);

L = 4;
xi = interp(x,L);

M = 3;
xd = decimate(x,M);

figure;
subplot(3,1,1);
plot(t,x);xlabel('Time (s)');ylabel('Amplitude');
title('Original Signal');grid on;
subplot(3,1,2);
stem(0:length(xi)-1,xi);xlabel('Time (s)');ylabel('Amplitude');
title('4-Factor Interpolated Signal');grid on;
subplot(3,1,3);stem(0:length(xd)-1,xd);xlabel('Time (s)');ylabel('Amplitude');
title('3-Factor Decimated Signal');grid on;
```




```
%9. Let X(k) be the 8-point DFT of a 4-point sequence x(n) = {3 ? , ?3, 3,-3}. Let Y (k) be the 8-point
%   DFT of a sequence y(n). Determine y(n) when Y(k) = X(k + 1)8

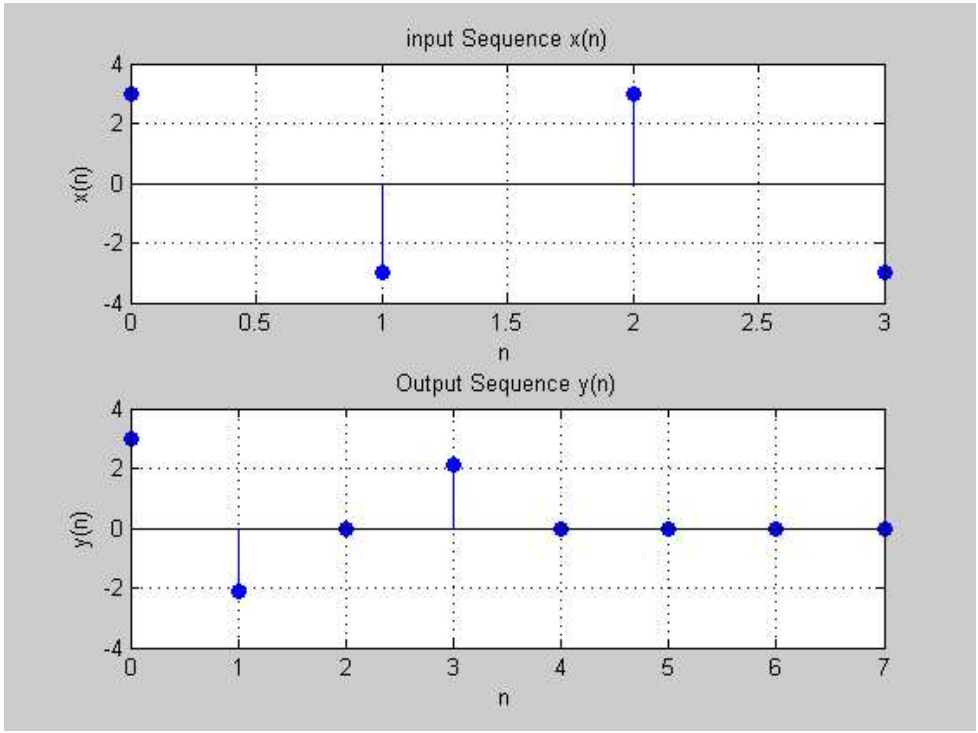
clc;clear;close all;

x = [3 -3 3 -3];
X = fft(x,8);

Y = circshift(X,[0 -1]);
y = ifft(Y,8);

n = 0:7;

figure;subplot(211);
stem(0:length(x)-1,x,'filled');
xlabel('n');
ylabel('x(n)');
title('input Sequence x(n)');
grid on;
subplot(212);
stem(n,real(y),'filled');
xlabel('n');
ylabel('y(n)');
title('Output Sequence y(n)');
grid on;
```




```
%10. Let X(k) be the 4-DFT of a x(n). Given X(k-2)4 = [3, -2-3j , 10, -2+3j] . Evaluate x(n-1) and
%    observe magnitude & angle plots of the spectrum X(k).

clc;
clear;
close all;

Y = [3, -2-3j, 10, -2+3j];

X = circshift(Y,[0 2]);

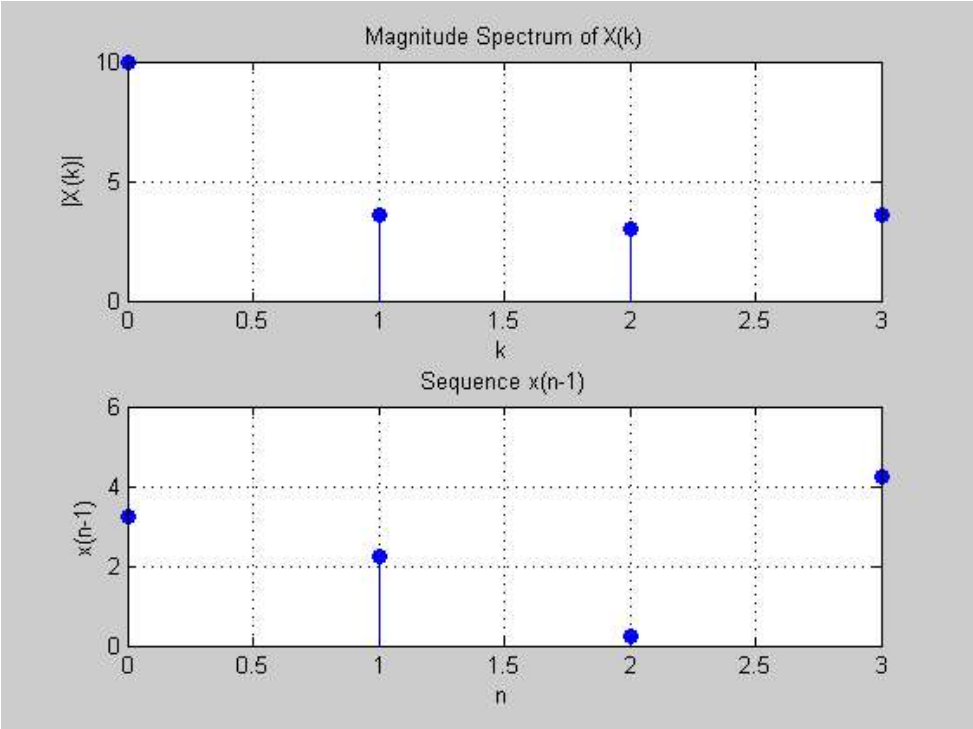
x = ifft(X,4);

y = circshift(x,[0 1]);

k = 0:3;
n = 0:3;

% Magnitude spectrum
figure;subplot(211);
stem(k,abs(X),'filled');
xlabel('k');
ylabel('|X(k)|');
title('Magnitude Spectrum of X(k)');
grid on;

% Plot x(n-1)
subplot(212);
stem(n,real(y),'filled');
xlabel('n');
ylabel('x(n-1)');
title('Sequence x(n-1)');
grid on;
```




```
%11. Design a lowpass Digital Butterworth filter, using Bilinear transformation, sampled at 2kHz. The
%    passband edge frequency is 200 Hz and the stopband edge frequency is 600 Hz. The attenuation in
%    passband is 1.938 dB and minimum stopband attenuation is 13.98 dB . Plot the frequency response
%    of your designed filter and obtain pole -zero plot of digital filter.

clc;
clear;
close all;

fs = 2000;
fp = 200;
fst = 600;
Ap = 1.938;
As = 13.98;

Wp = fp/(fs/2);
Ws = fst/(fs/2);

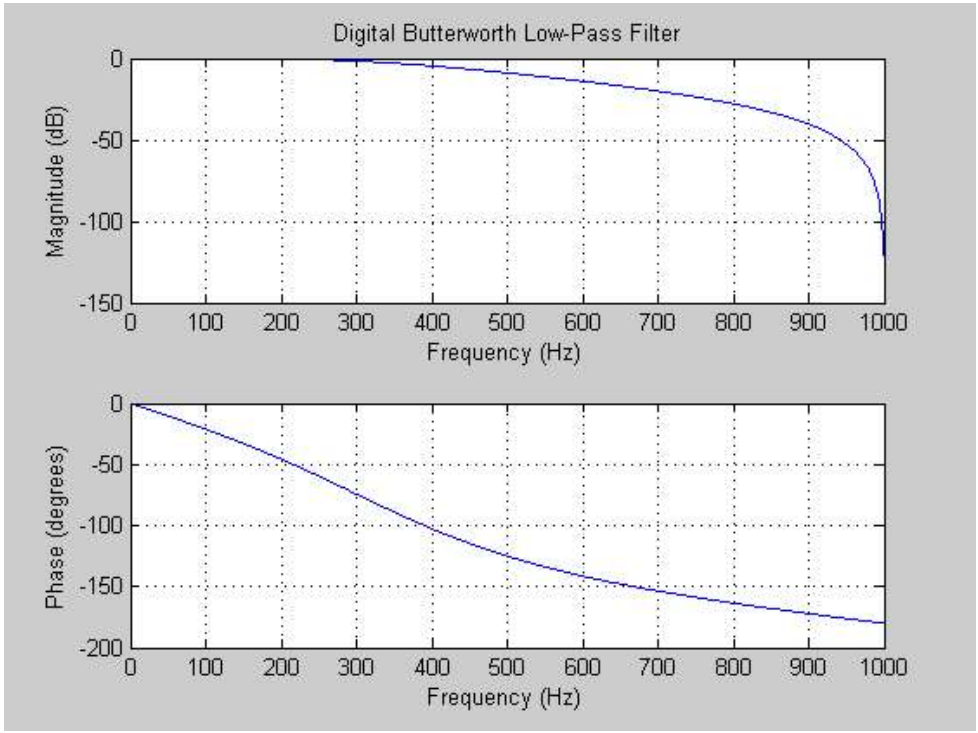
[N,Wn] = buttord(Wp,Ws,Ap,As);

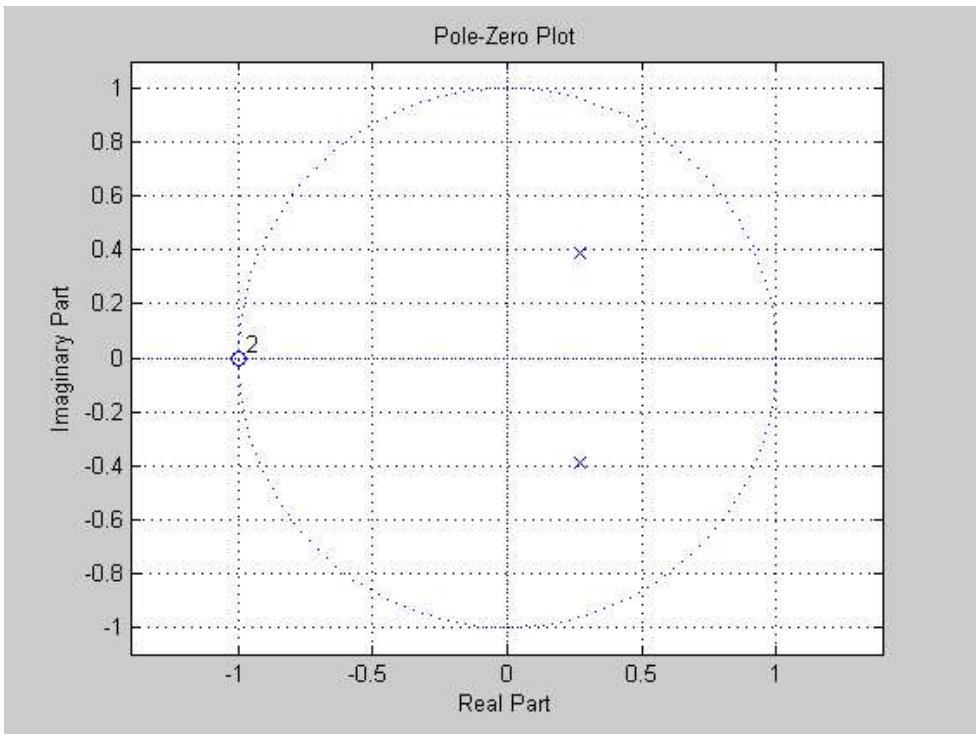
[b,a] = butter(N,Wn,'low');

H = tf(b,a,1/fs);

figure;
freqz(b,a,1024,fs);
title('Digital Butterworth Low-Pass Filter');
grid on;

% Pole-zero plot
figure;
zplane(b,a);
title('Pole-Zero Plot');
grid on;
```






```
%12. Let LTI systems with impulse responses h1(n) & h2(n), Consider the new system composed of the
%     cascading of h1(n) & h2(n) resulting the impulse response of the new system h3(n).
%     h1(n)= -0.04?(n)+ 0.04?(n-1)+ 0.3?(n-2)-0.6?(n-3)+ 0.3?(n-4) +0.04?(n-5) -0.04?(n-6),
%     h2(n)= 0.09?(n)- 0.12?(n-1)-0.5?(n-2)-0.5?(n-3)+ 0.12?(n-4) +0.09?(n-5)
%     i) Evaluate h3(n)= h1(n)* h2(n)* 3?(n) & obtain transfer function and its magnitude response.
%     ii) Evaluate h3(n)= h1(n)* h2(n)* ?(n-1) & obtain transfer function and its magnitude response

clc;clear;close all;

h1 = [-0.04 0.04 0.3 -0.6 0.3 0.04 -0.04];
h2 = [0.09 -0.12 -0.5 -0.5 0.12 0.09];

% Convolution of h1 and h2
h = conv(h1,h2);
```

```
% (i) h3(n) = h1(n)*h2(n)*3delta(n)
h3a = 3*h;

% (ii) h3(n) = h1(n)*h2(n)*delta(n-1)
h3b = [0 h];

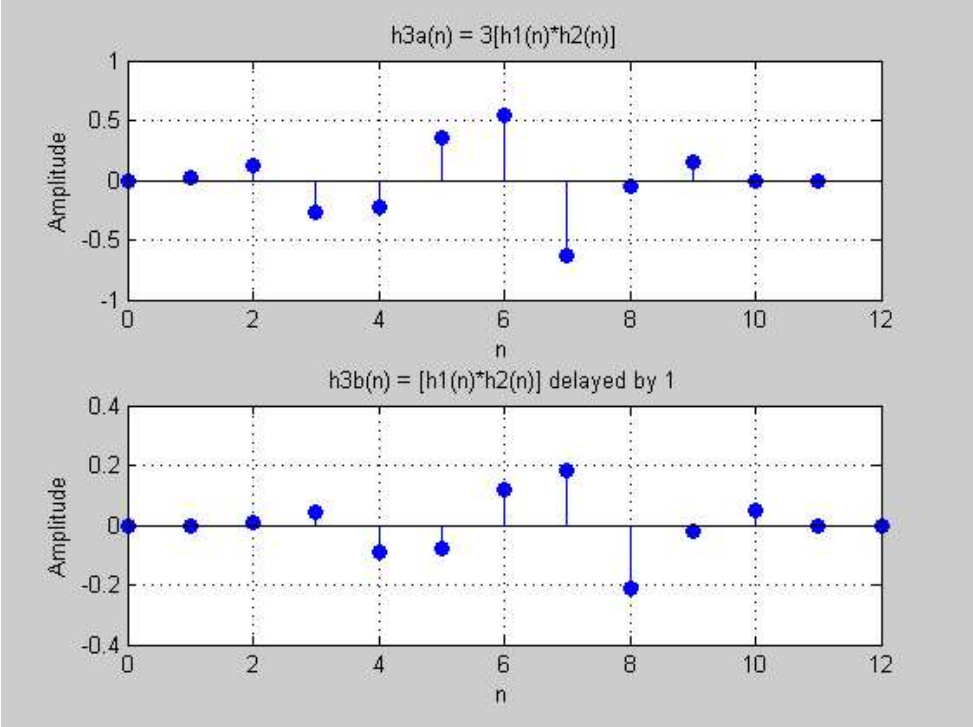
H3a = tf(h3a,1,1);
H3b = tf(h3b,1,1);

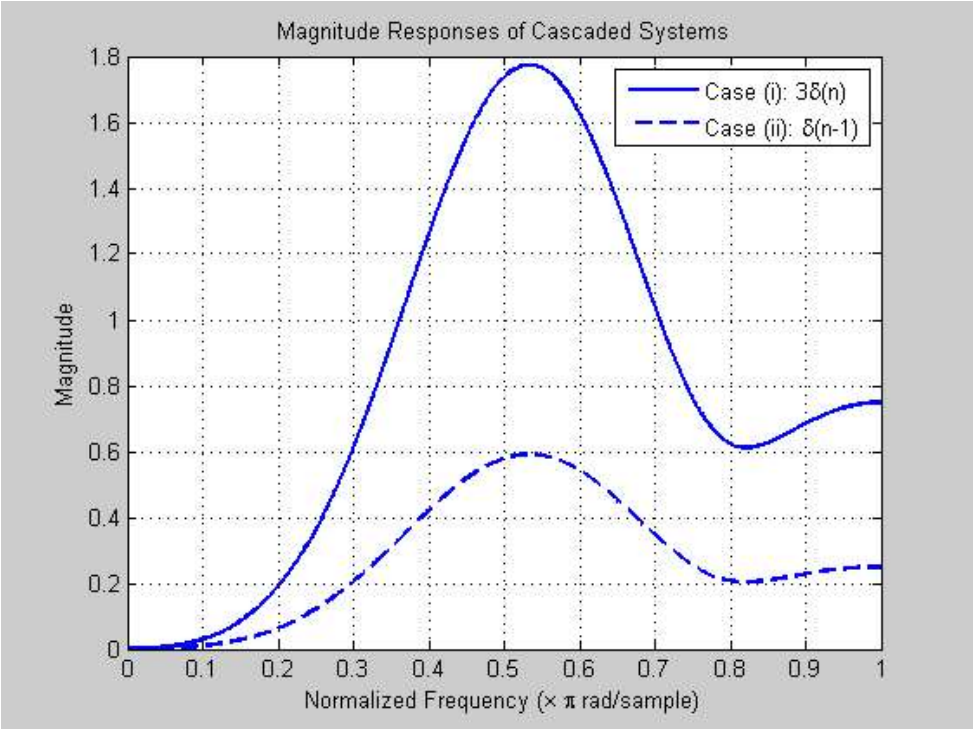
% Plot impulse responses
figure;
subplot(2,1,1);
stem(0:length(h3a)-1,h3a,'filled');
title('h3a(n) = 3[h1(n)*h2(n)]');
xlabel('n');
ylabel('Amplitude');
grid on;

subplot(2,1,2);
stem(0:length(h3b)-1,h3b,'filled');
title('h3b(n) = [h1(n)*h2(n)] delayed by 1');
xlabel('n');
ylabel('Amplitude');
grid on;

% Magnitude responses
[H1,w] = freqz(h3a,1,1024);
[H2,w] = freqz(h3b,1,1024);

figure;
plot(w/pi,abs(H1),'LineWidth',1.5);
hold on;
plot(w/pi,abs(H2),'--','LineWidth',1.5);
xlabel('Normalized Frequency (\times \pi rad/sample)');
ylabel('Magnitude');
title('Magnitude Responses of Cascaded Systems');
legend('Case (i): 3\delta(n)','Case (ii): \delta(n-1)');
grid on;
```






```
%13. Let X1(k) & X2(k) are the DFTs of x1(n) & x2(n) respectively.
%    Given X1(-k)4=[7 -1 -1 -1] & X2(k-1)4=[0 6 0 -2]. Evaluate the linear convolution and 6-point
%    circular convolution of x1(n) & x2(n) using X1(k) & X2(k).

clc;clear;close all;
A = [7 -1 -1 -1];
B = [0 6 0 -2];

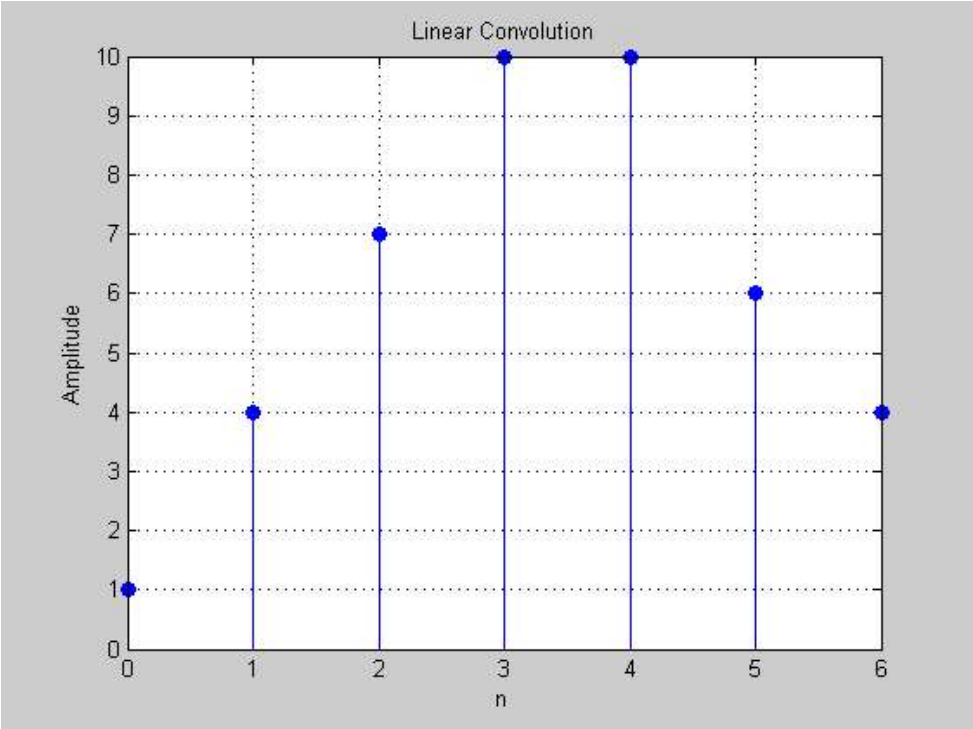
X1 = A;
X2 = circshift(B,[0 -1]);

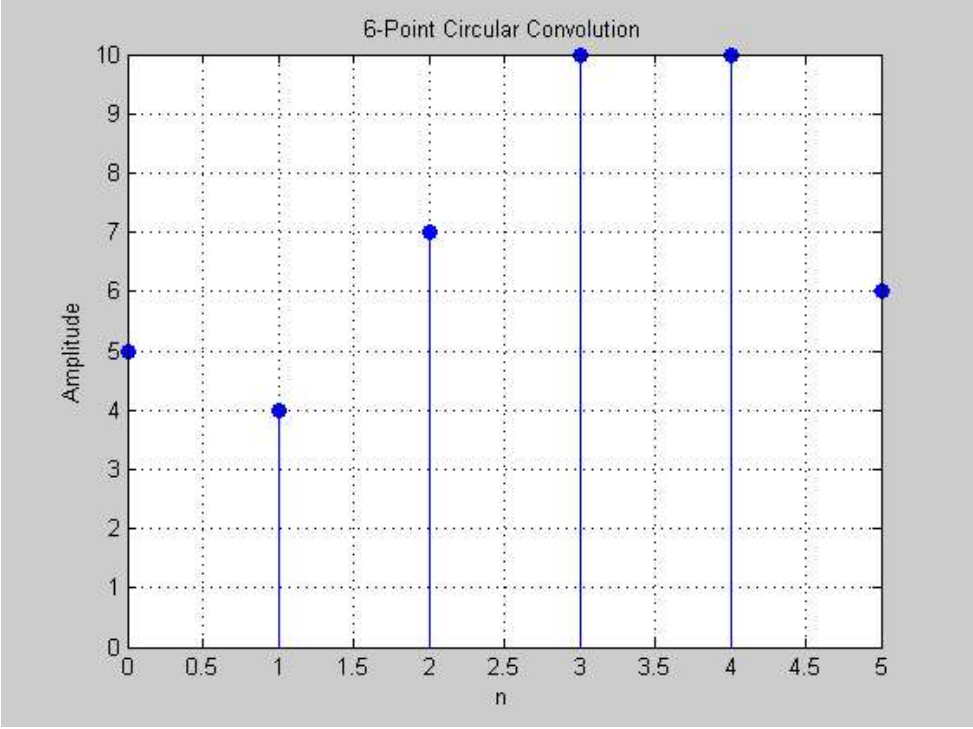
x1 = ifft(X1,4);
x2 = ifft(X2,4);
y_linear = conv(x1,x2);

X1_6 = fft(x1,6);
X2_6 = fft(x2,6);
y_circular = ifft(X1_6.*X2_6,6);

figure;
stem(0:length(y_linear)-1,real(y_linear),'filled');
xlabel('n');
ylabel('Amplitude');
title('Linear Convolution');
grid on;

% Plot circular convolution
figure;
stem(0:5,real(y_circular),'filled');
xlabel('n');
ylabel('Amplitude');
title('6-Point Circular Convolution');
grid on;
```






```
%14. Let X(k) be the 4-DFT of x(n)= {5,4,3,4},
%    i) Evaluate X(k+2)4 & X(k)4 and sketch their spectrum

clc;clear;close all;

x = [5 4 3 4];
X = fft(x,4);
X_shift = circshift(X,[0 -2]);

k = 0:3;

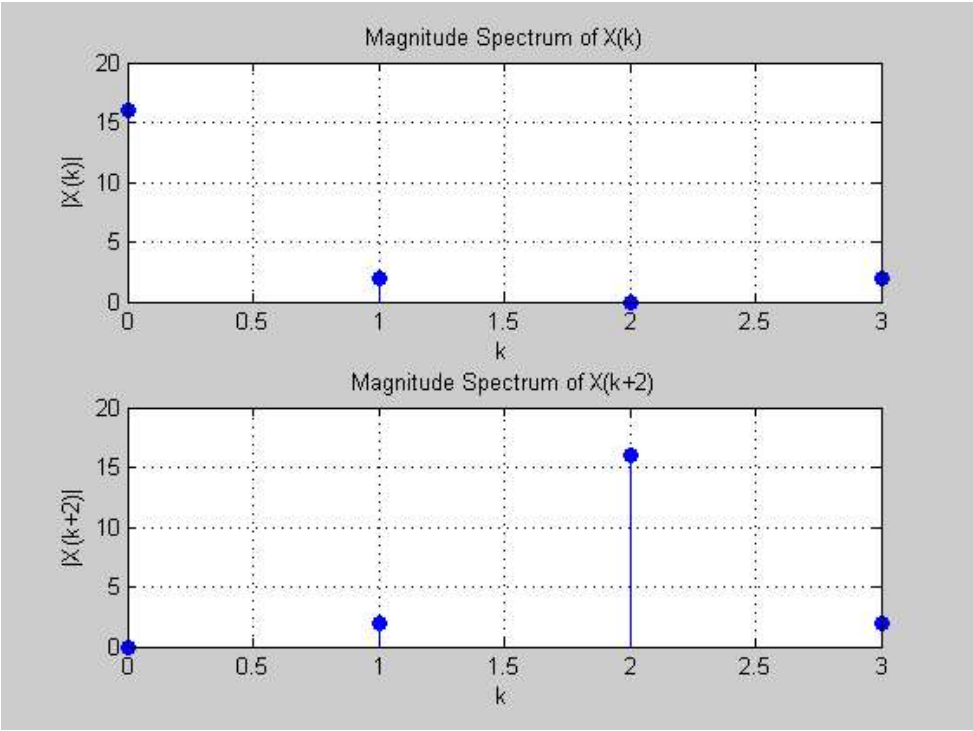
figure;
subplot(2,1,1);
stem(k,abs(X),'filled');
title('Magnitude Spectrum of X(k)');
```

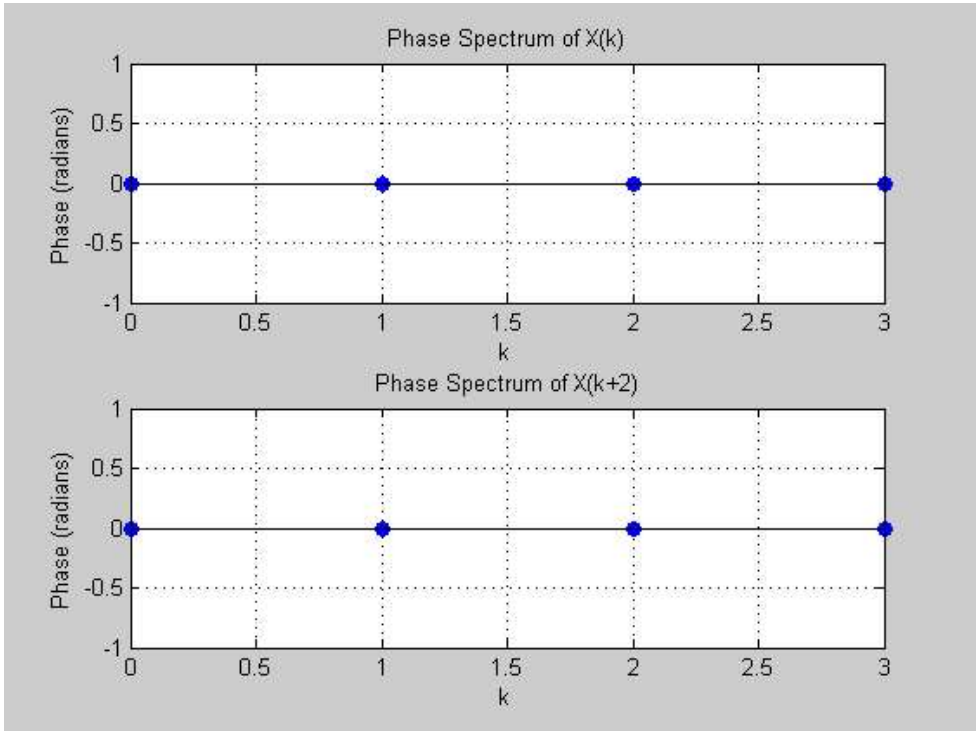
```
xlabel('k');
ylabel('|X(k)|');
grid on;

subplot(2,1,2);
stem(k,abs(X_shift),'filled');
title('Magnitude Spectrum of X(k+2)');
xlabel('k');
ylabel('|X(k+2)|');
grid on;

% Phase spectra
figure;
subplot(2,1,1);
stem(k,angle(X),'filled');
title('Phase Spectrum of X(k)');
xlabel('k');
ylabel('Phase (radians)');
grid on;

subplot(2,1,2);
stem(k,angle(X_shift),'filled');
title('Phase Spectrum of X(k+2)');
xlabel('k');
ylabel('Phase (radians)');
grid on;
```



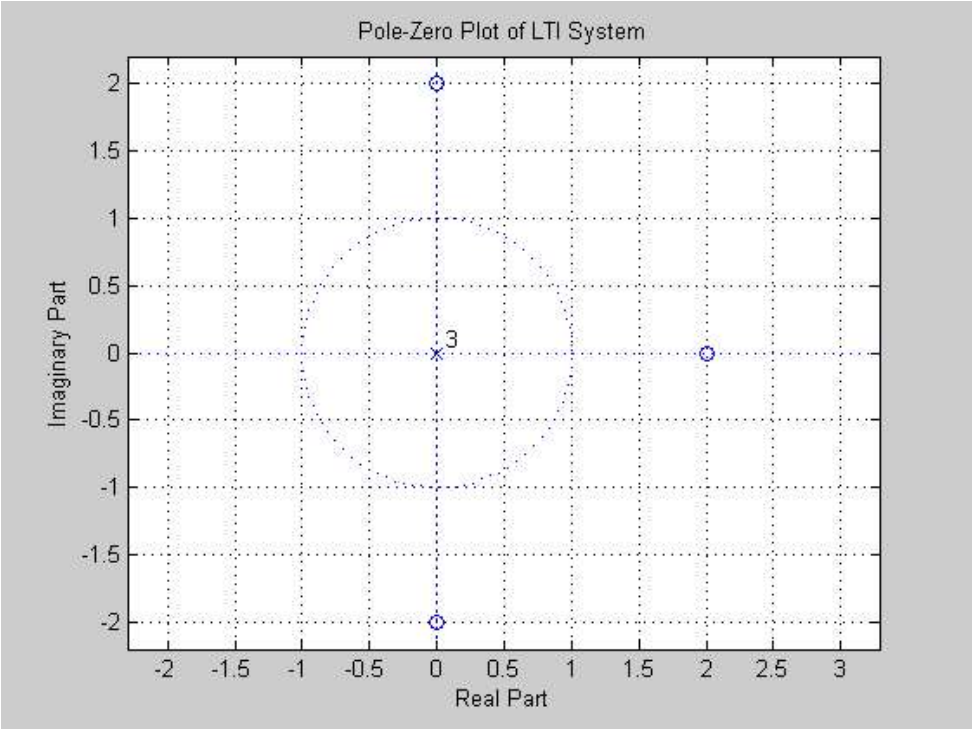



```
%15. Obtain pole-zero plot corresponds to transfer function of LTI system characterized by difference
%    equation given as,y(n) = ?3 to k=0 ( (?2)^ k * x(n-k) )
% y(n)=x(n)-2x(n-1)+4x(n-2)-8x(n-3)
% H(z)={z^3-2z^2+4z-8}/{z^3}
clc;clear;close all;

b = [1 -2 4 -8];
a = [1 0 0 0];

H = tf(b,a,1);

% Pole-zero plot
figure;
zplane(b,a);
title('Pole-Zero Plot of LTI System');
grid on;
```




```
%16. Let step response of the DT-LTI system is s(n)=2?(n+1)+ ?(n)+?(n-1). Let y(n) be the output of the
%    system for an input x(n) = 0.5^n u(n) .Determine the value of y(1) ?
%    h(n)=s(n)-s(n-1)

clc;clear;close all;
n = -3:5;

s = 2*(n == -1) + (n == 0) + (n == 1);

s1 = 2*(n-1 == -1) + (n-1 == 0) + (n-1 == 1);

h = s - s1;

x = (0.5).^n .* (n >= 0);

% Calculate output using convolution
y = conv(x,h);
ny = (n(1)+n(1)):(n(end)+n(end));

% Find y(1)
y1 = y(ny == 1);

disp('y(1) = ');
disp(y1);


y(1) =
     0



%17. Let X(k)=k+1; 0 ? k ? 7 be the 8-point DFT of a sequence x(n).
%    Obtain and sketch x(n) and evaluate the value of ?3 to k=0 x(2n).

clc;clear;close all;

X = 1:8;

x = ifft(X,8);

n = 0:7;

figure;
subplot(2,1,1);
stem(n,real(x),'filled');
```

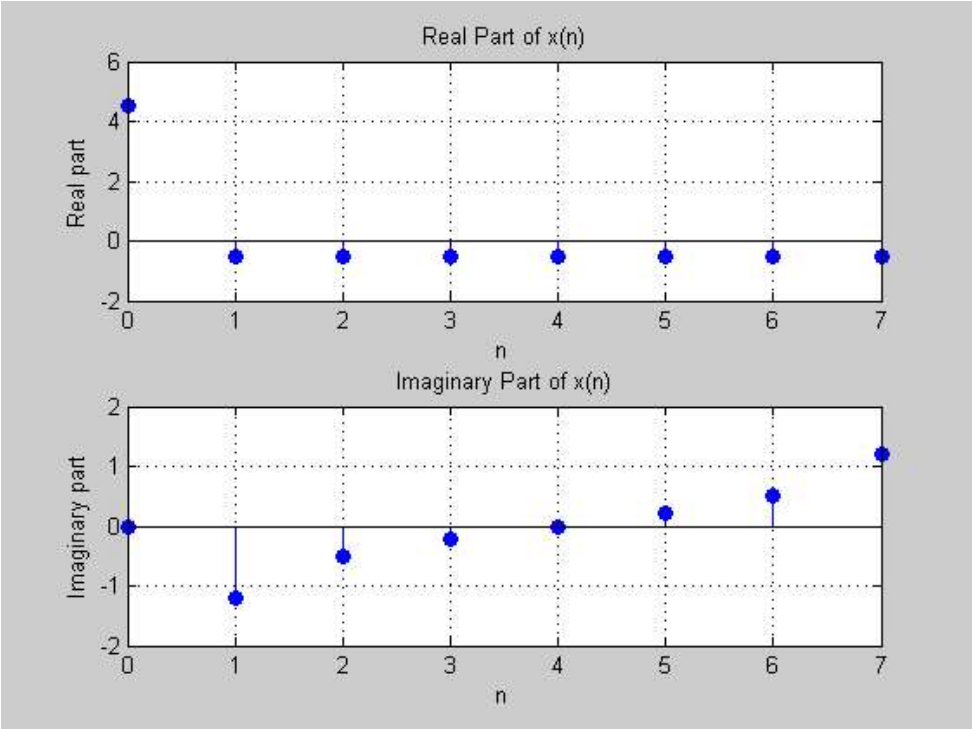
```
xlabel('n');
ylabel('Real part');
title('Real Part of x(n)');
grid on;

subplot(2,1,2);
stem(n,imag(x),'filled');
xlabel('n');
ylabel('Imaginary part');
title('Imaginary Part of x(n)');
grid on;

% Calculate sum of x(2k), k = 0 to 3
S = sum(x(1:2:7));

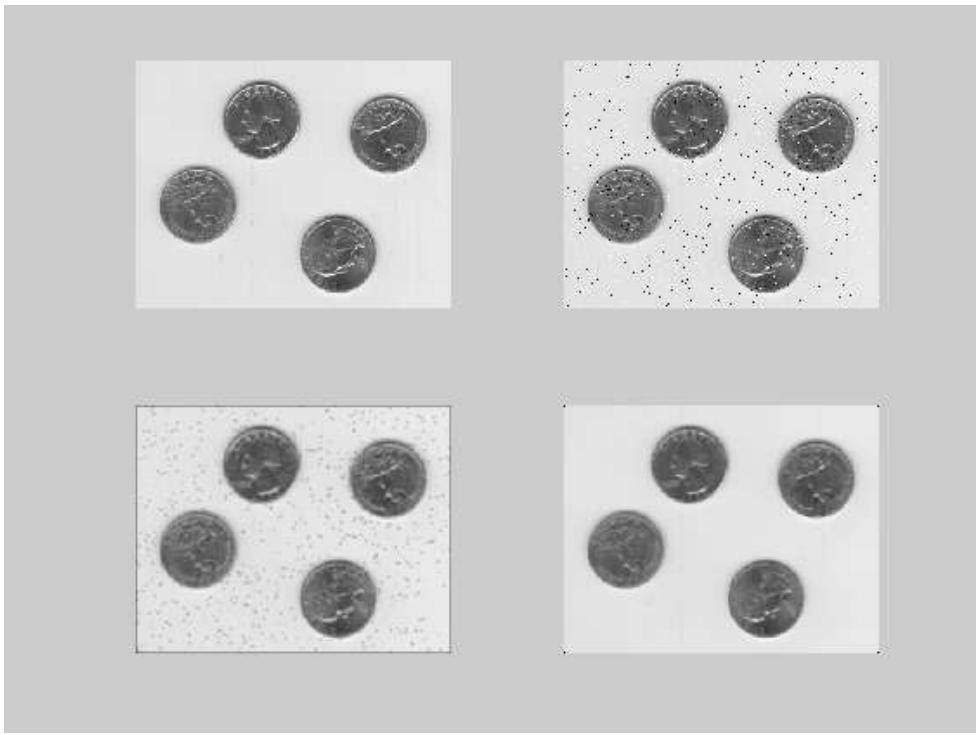
disp('Sum of x(2k), k=0 to 3 = ');
disp(S);


Sum of x(2k), k=0 to 3 =
     3
```




```
%18. Develop a spatial filtering simulation to process an image corrupted by noise:
%     a) Manually add Salt-and-Pepper noise to 5% of an input image's pixels.
%     b) Implement a 3×3 Linear Average (Box) Filter and a 3×3 Non-linear Median Filter to clean the nois

clc;clear;close all;
I = imread('eight.tif');
figure,subplot(221);imshow(I);
J = imnoise(I,'salt & pepper',0.02);
subplot(222); imshow(J);
K = filter2(fspecial('average',3),J)/255;
subplot(223); imshow(K);
L = medfilt2(J,[3 3]);
subplot(224); imshow(L);
```



```
%19. A discrete-time LTI system is characterized by the following linear constant-coefficient difference
%equation: y(n)?0.75y(n?1)+0.125y(n?2)=x(n)+x(n?1)
%i) Obtain impulse response and determine whether the system is stable or not
% H(z)={1+z^{-1}}/{1-0.75z^{-1}+0.125z^{-2}}
clc;clear;close all;

b = [1 1];
a = [1 -0.75 0.125];
[h,n] = impz(b,a,20);

figure;
stem(n,h,'filled');
xlabel('n');
ylabel('h(n)');
title('Impulse Response');
grid on;

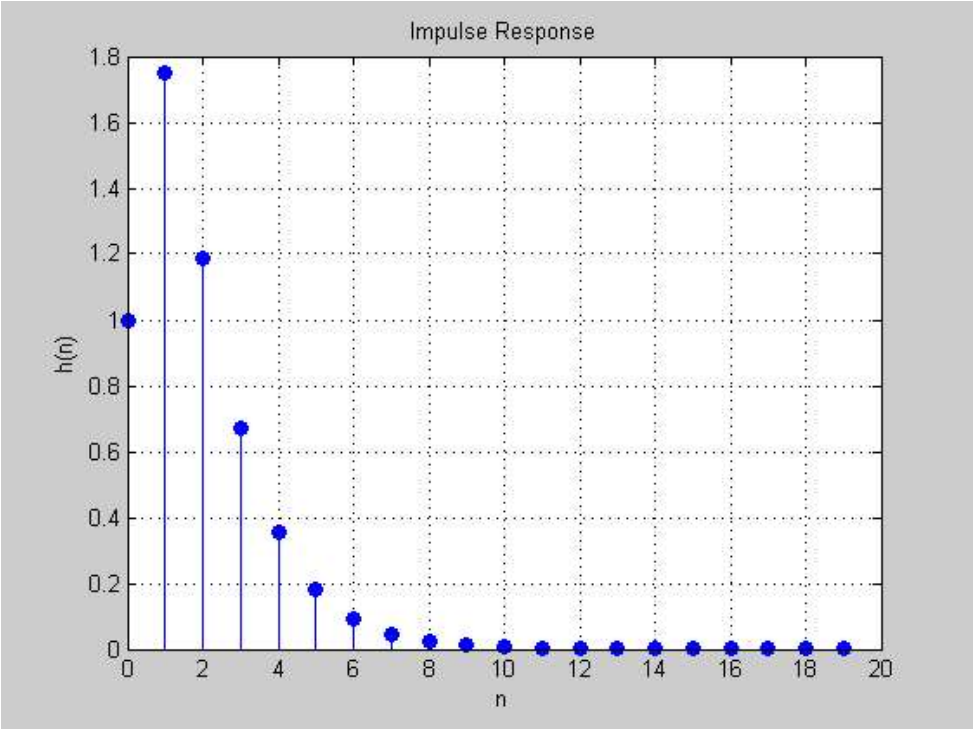
figure;
zplane(b,a);
title('Pole-Zero Plot');
grid on;

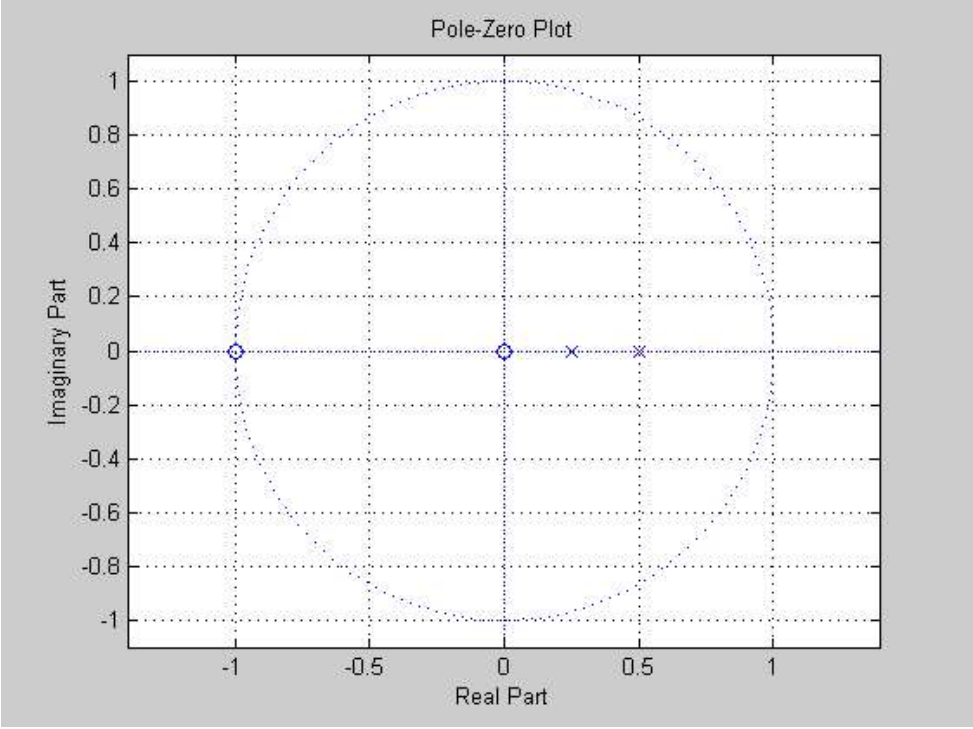
% Check stability
p = roots(a);
disp('Poles:');
disp(p);

if all(abs(p) < 1)
    disp('System is Stable');
else
    disp('System is Unstable');
end
```

```
Poles:
    0.5000
    0.2500

System is Stable
```






```
%20. Let step response of the DT-LTI system is s(n)=2?(n)+ ?(n-1)+?(n-2). Let y(n) be the output of the
%    system for an input x(n)=(0.3)^n u(n). Determine the value of y(1) ?

clc;clear;close all;
n = -3:5;
deln = (n == 0);
s = 2*(n == 0) + (n == 1) + (n == 2);
s1 = 2*(n == 1) + (n == 2) + (n == 3);
h = s - s1;

x = (0.3).^n .* (n >= 0);

% Calculate output using convolution
y = conv(x,h);
ny = (n(1)+n(1)):(n(end)+n(end));
```

```
% Find y(1)
y1 = y(ny == 1);

disp('y(1) = ');
disp(y1);
```

```
y(1) =
   -0.4000
```

Published with MATLAB® 7.5